

DESIGN & ANALYSIS OF ALGORITHMS

COMPREHENSIVE EXAM & VIVA REVISION GUIDE

▲ UNIT III Graph Algorithms ▲

COVERS:

BFS · DFS · SCC · BiCC · Dijkstra · Bellman-Ford · Floyd-Warshall · MST · Kruskal · Prim

Knapsack · Matrix Chain · TSP · Boyer-Moore · KMP · Backtracking · NP-Complete · SAT

All Algorithms · All Formulas · All Complexities

Exam Preparation Resource

UNIT III — GRAPHS

1. Graph Representations

A graph $G = (V, E)$ can be represented computationally in two standard ways: Adjacency Lists and Adjacency Matrices. Both representations apply to directed and undirected graphs.

Adjacency-List Representation

Consists of an array Adj of $|V|$ lists. For each $u \in V$, the list $Adj[u]$ contains all vertices v such that there is an edge $(u, v) \in E$.

- **Structure:** Array of $|V|$ linked lists, one per vertex.
- **Space Complexity:** $\Theta(V + E)$ — compact, especially for sparse graphs ($|E| \ll |V|^2$).
- **Directed Graph:** Sum of all list lengths = $|E|$.
- **Undirected Graph:** Sum of all list lengths = $2|E|$ (each edge (u,v) appears in both $Adj[u]$ and $Adj[v]$).
- **Disadvantage:** Traversal of a vertex's neighbors is slower than matrix lookup.
- **Best For:** Sparse graphs.

Adjacency-Matrix Representation

A $|V| \times |V|$ matrix $A = (a_{ij})$ where vertices are ordered $1, 2, \dots, |V|$ and $a_{ij} = 1$ if $(i, j) \in E$, else 0 .

- **Space Complexity:** $\Theta(V^2)$ — independent of number of edges.
- **Edge Check:** $O(1)$ constant time — simple to implement.
- **Symmetry:** Undirected graphs produce a symmetric matrix ($A = A^T$); store only upper triangle to save space.
- **Weighted Graphs:** Store edge weights directly in the matrix cells.
- **Best For:** Dense graphs ($|E| \approx |V|^2$).
- **Disadvantage:** $\Theta(V^2)$ space regardless of edge count — wasteful for sparse graphs.

DESIGN & ANALYSIS OF ALGORITHMS

COMPREHENSIVE EXAM & VIVA REVISION GUIDE

Quick Comparison

UNIT III: Graph Algorithms

COVERS

Feature	Adjacency List	Adjacency Matrix
Best For	Sparse Graphs ($ E \ll V ^2$)	Dense Graphs ($ E \approx V ^2$)
Space	$\Theta(V + E)$	$\Theta(V^2)$
Edge Check	$O(\text{degree of } u)$	$O(1)$
Memory Overhead	Low for sparse	High for sparse
Traversal (BFS/DFS)	Efficient	Efficient

■ EXAM QUESTIONS

1. Compare Adjacency Matrix and Adjacency List representations. Which is better for a sparse graph and why?
2. Write the defining matrix formula for an Adjacency Matrix representing an unweighted graph $G = (V, E)$.
3. What is the space complexity of the Adjacency List representation for a directed graph with $|V|$ vertices and $|E|$ edges?

2. Breadth-First Search (BFS)

BFS systematically explores edges to discover every vertex reachable from a source s , computing the shortest distance (fewest edges) to each reachable vertex. It expands the frontier uniformly across its breadth.

All Algorithms · All Formulas · All Complexities

Exam Preparation Resource

Concepts & Mechanism

- **Frontier Expansion:** Discovers all vertices at distance k before moving to distance $k+1$.
- **Data Structure:** Uses a FIFO Queue (Q) to manage gray (frontier) vertices.
- **Attributes per Vertex:** $u.color$ (state), $u.d$ (distance from source s), $u.\pi$ (predecessor/parent).

Coloring System (3 States)

WHITE	Undiscovered — vertex not yet seen by BFS.
GRAY	Discovered — vertex seen, but neighbors still being explored (on the frontier).
BLACK	Fully explored — all adjacent vertices discovered and processed.

Algorithm Steps

1. Initialize all vertices: color = WHITE, $d = \infty$, $\pi = \text{NIL}$. Set source s : color = GRAY, $d = 0$.
2. Enqueue s into Q .
3. While Q is not empty: Dequeue u from Q .
4. For each neighbor v in $\text{Adj}[u]$: If $v.\text{color} == \text{WHITE} \rightarrow$ set $v.\text{color} = \text{GRAY}$, $v.d = u.d + 1$, $v.\pi = u$, enqueue v .
5. Set $u.\text{color} = \text{BLACK}$ (fully processed).

Time Complexity Analysis

- **Enqueue/Dequeue operations:** Each vertex enqueued and dequeued at most once $\rightarrow O(V)$.
- **Adjacency List scanning:** Each adjacency list $\text{Adj}[u]$ scanned once when u is dequeued $\rightarrow O(E)$ total.
- **Total Time Complexity:** $O(V + E)$

■ EXAM QUESTIONS

1. Explain the role of the three colors (White, Gray, Black) used in BFS. Why is a FIFO queue used?
2. What does $u.d$ store in BFS, and what does it represent mathematically?
3. Trace BFS on a graph starting from vertex s . Label $u.d$ and $u.\pi$ for each vertex.

3. Depth-First Search (DFS)

DFS follows the strategy of searching '**deeper**' in the graph whenever possible. It explores edges out of the most recently discovered vertex v that still has unexplored edges. Once all of v 's edges are explored, it **backtracks** to the vertex from which v was discovered.

Key Concepts

- **DFS Forest:** Unlike BFS (one tree), DFS produces a depth-first forest comprising multiple trees if the graph is disconnected.
- **Timestamps:** Each vertex u has two timestamps recorded:
 - **$u.d$ (Discovery time):** When u is first discovered (grayed).
 - **$u.f$ (Finishing time):** When u is blackened (all edges fully examined).
- **Invariant:** For every vertex u , $u.d < u.f$ (discovery always precedes finishing).
- **Coloring:** Same three-color scheme as BFS — WHITE, GRAY, BLACK.

Edge Classification in DFS

DFS classifies every edge (u, v) encountered during exploration into one of four categories:

Edge Type	Description	Target Color
Tree Edge	Edges in the DFS forest. (u, v) is a tree edge if v was first discovered via u .	WHITE
Back Edge	Connects vertex u to an ancestor v in a DFS tree. Includes self-loops.	GRAY

Forward Edge	Non-tree edges connecting u to a descendant v ($u.d < v.d$, v is BLACK).	BLACK ($u.d < v.d$)
Cross Edge	All other edges — between vertices in same or different DFS trees ($u.d > v.d$).	BLACK ($u.d > v.d$)

■ **Key Fact:** A **Back Edge** indicates the presence of a **cycle** in a directed graph. In undirected graphs, only Tree and Back edges can appear.

Time Complexity

- **Total Time Complexity:** $\Theta(V + E)$
- Each vertex is colored and timestamped exactly once ($O(V)$), and each edge is examined once ($O(E)$).

Applications of DFS

- **Topological Sort** — Linear ordering of DAG vertices.
- **Strongly Connected Components (SCC)** — Finding maximally connected subgraphs.
- **Biconnected Components & Articulation Points** — Finding bridges and cut vertices.

■ EXAM QUESTIONS

1. Define Tree, Back, Forward, and Cross edges in DFS. Which type indicates a cycle in a directed graph?
2. What are timestamps $u.d$ and $u.f$ in DFS? What is the relationship between them?
3. What is the time complexity of DFS? Justify.

4. Topological Sorting of Acyclic Graphs (DAG)

A **topological sort** of a Directed Acyclic Graph (DAG) $G = (V, E)$ is a linear ordering of all vertices such that if there is a directed edge (u, v) , then **u appears before v** in the ordering. Topological sorting is **impossible** if the graph contains a cycle.

Algorithm via DFS

1. Call DFS(G) to compute finishing times $v.f$ for every vertex v .
2. As each vertex finishes (is blackened), insert it onto the FRONT of a linked list.
3. Return the linked list — vertices appear in decreasing order of their finishing times.
(Vertex with highest $u.f$ appears first in the topological order.)

Time Complexity: $O(V + E)$ — same as DFS.

Applications

- **Dependency Resolution:** Schedule tasks such that prerequisites come first (e.g., course prerequisites, build systems).
- **Longest Path in a DAG:** Can be computed efficiently after topological ordering.
- **Event Scheduling:** Ordering events with partial ordering constraints.

■ EXAM QUESTIONS

1. How does DFS assist in computing the topological sort of a DAG? Describe the algorithm.
2. Why is topological sorting impossible on a graph containing a cycle?
3. What is the time complexity of topological sort and why?

5. Strongly Connected Components (SCC)

A **Strongly Connected Component** of a directed graph is a **maximally connected subgraph** where every vertex is reachable from every other vertex within that subgraph. Formally: for every pair (u, v) in the SCC, paths $u \rightarrow v$ and $v \rightarrow u$ both exist.

5-Step Algorithm to Find SCCs

Step 1	Perform DFS(G) on the original graph G to generate a DFS forest.
Step 2	Assign post-order numbers to vertices based on their finishing times u.f from Step 1.
Step 3	Compute G^T (Transpose Graph) — reverse all edge directions in G.
Step 4	Perform DFS(G^T), but always start new searches from the HIGHEST post-order numbered remaining vertex.
Step 5	Each tree in the resulting DFS forest of G^T is a separate SCC.

Time Complexity: $O(V + E)$ — Two DFS passes on adjacency list representation.

■ EXAM QUESTIONS

1. List all 5 algorithmic steps to find Strongly Connected Components (SCC) in a directed graph.
2. Define a Strongly Connected Component. Why must we perform the second DFS on the transposed graph G^T ?
3. What is the time complexity of Kosaraju's SCC algorithm?

6. Biconnected Components (BiCC) & Articulation Points

A **Biconnected Graph** is a connected undirected graph that remains connected even after the removal of any single vertex and its incident edges. A **Biconnected Component** is a maximally connected biconnected subgraph.

Articulation Point (AP)

A vertex **v** is an **Articulation Point** if its removal (along with all its incident edges) disconnects the graph into two or more non-empty components. Articulation points are also called **cut vertices**.

DFS Tree Edge Types Used

- **Tree Edges:** Solid edges forming the DFS tree (parent→child).
- **Back Edges:** Dashed edges connecting a descendant to an ancestor.

The Low Value — Core Formula

The **Low(w)** value is the **lowest discovery time** reachable from the subtree rooted at w using tree edges and at most one back edge upward.

$$\text{Low}(w) = \min \{ d[w], \min \{ \text{Low}[\text{child of } w] \}, \min \{ d[\text{back-edge endpoints from } w] \} \}$$

Calculation Order: Compute Low values bottom-up — start from leaf nodes and move toward the root (since parent's Low depends on children's Low values).

Three Conditions for a Vertex to be an AP

Root Node	The DFS root is an AP if and only if it has at least 2 children in the DFS tree.
Leaf Node	A leaf node is NEVER an articulation point (removing it cannot disconnect the graph).
Internal Node u	An internal node u is an AP if it has a child w such that: $d[u] \leq \text{Low}[\text{child}[w]]$ — meaning no descendant of w can reach above u via a back edge.

Example: In a sample graph with 10 nodes, nodes **2, 3, and 5** were identified as articulation points because they satisfied the condition $d[u] \leq \text{Low}[\text{child}[u]]$.

■ EXAM QUESTIONS

1. Define an Articulation Point. State the mathematical condition for an internal DFS node to be an AP.
2. What is the $\text{Low}(w)$ value? Write its formula and explain how it is computed.
3. When is the root of a DFS tree considered an Articulation Point?

7. Shortest Path Algorithms — Overview & Relaxation

All shortest-path algorithms use the **Relaxation** technique. Attribute **v.d** stores the shortest-path estimate from source *s* to *v*, and **v.π** stores the predecessor vertex on the shortest path.

The Relaxation Step

```
RELAX(u, v, w): if v.d > u.d + w(u, v): v.d = u.d + w(u, v) v.π = u
```

Relaxing edge (*u*, *v*) checks whether the current shortest path to *v* can be improved by going through *u*. If yes, update *v.d* and set *v.π* = *u* (*u* is now *v*'s predecessor).

A. Bellman-Ford Algorithm (Dynamic Programming)

Solves the **Single-Source Shortest Path (SSSP)** problem. More versatile than Dijkstra — it can handle **negative edge weights** and can **detect negative-weight cycles**.

Key Characteristics

- **Graph Type:** Directed or undirected, allows negative weights.
- **Strategy:** Dynamic Programming — relaxes ALL edges exactly $|V| - 1$ times.
- **Negative Cycle Detection:** If a negative cycle is reachable from the source, no shortest path exists (you can loop indefinitely to decrease distance).

Algorithm Steps

1. Initialize: source.d = 0, all other vertices.d = ∞ , all *v.π* = NIL.
2. Iterative Relaxation: Repeat $|V| - 1$ times → Relax EVERY edge (*u*, *v*) in the graph.
3. Negative Cycle Check: Relax all edges ONE MORE TIME (pass $|V|$). If any *v.d* still decreases → return FALSE (negative cycle exists).
4. If no decrease detected → return TRUE (shortest paths are valid).

Time Complexity Analysis

- Initialization: $\Theta(V)$
- Edge relaxation passes: $(|V| - 1) \times \Theta(E) = \mathbf{O(VE)}$
- Final check pass: $\Theta(E)$
- **Total: $\mathbf{O(VE)}$**

B. Dijkstra's Algorithm (Greedy Approach)

Solves **SSSP** on graphs with **non-negative edge weights only**. Uses a Greedy strategy — at each step it selects the vertex with the minimum distance estimate.

Key Characteristics

- **Graph Type:** Directed or undirected with non-negative weights.
- **Constraint:** FAILS if negative edge weights are present.
- **Strategy:** Maintains a set *S* of vertices with finalized shortest paths. Repeatedly extracts the minimum-distance vertex from a priority queue *Q*.

Algorithm Steps

1. Initialize: source.d = 0, all others.d = ∞ , all *v.π* = NIL.
2. Place all vertices into min-priority queue *Q* keyed by *v.d*.
3. While *Q* is not empty: Extract vertex *u* = EXTRACT-MIN(*Q*). Add *u* to *S*.
4. For each neighbor *v* of *u*: RELAX(*u*, *v*, *w*). If *v.d* decreased, update *Q* (decrease-key).

Time Complexity — Heap Analysis

Implementation	EXTRACT-MIN Cost	RELAX / Decrease-Key	Total Complexity
----------------	------------------	----------------------	------------------

Array (simple)	$O(V) \times V$ calls	$O(1) \times E$ calls	$O(V^2 + E) \approx O(V^2)$
Binary Min-Heap	$O(\log V) \times V$ calls	$O(\log V) \times E$ calls	$O(E \log V)$
Fibonacci Heap	$O(\log V)$ amortized	$O(1)$ amortized	$O(E + V \log V)$

Binary Heap Derivation: EXTRACT-MIN runs V times $\rightarrow V \times O(\log V)$. Decrease-key (during RELAX) runs at most E times $\rightarrow E \times O(\log V)$. Total = $O(V \log V + E \log V) = \mathbf{O(E \log V)}$.

C. Floyd-Warshall Algorithm — All-Pairs Shortest Path (APSP)

Unlike Dijkstra and Bellman-Ford (which are SSSP), Floyd-Warshall finds shortest paths **between every pair of vertices**. Uses Dynamic Programming with intermediate vertices.

Key Characteristics

- **Type:** All-Pairs Shortest Path (APSP) using Dynamic Programming.
- **Handles:** Negative edges (allowed). Does NOT work with negative cycles.
- **Strategy:** Considers each vertex k as a potential 'intermediate' node on the path from i to j .

Recurrence Formula

$$d(k)[i][j] = w(i,j) \text{ if } k = 0 = \min(d(k-1)[i][j], \text{ if } k \geq 1 \text{ } d(k-1)[i][k] + d(k-1)[k][j])$$

Where $d^{(k)}[i][j]$ = shortest path from i to j using only intermediate vertices from $\{1, 2, \dots, k\}$.

Algorithm Steps

1. Initialize: $n \times n$ matrix $D(0)$ with edge weights. Set $D[i][i] = 0$, $D[i][j] = \infty$ if no direct edge.
2. Three nested loops:
For $k = 1$ to V (each intermediate vertex):
For $i = 1$ to V (each source):
For $j = 1$ to V (each destination):
 $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$

- **Time Complexity:** $\Theta(V^3)$ — Three nested loops each running V times.
- **Space Complexity:** $\Theta(V^2)$ — To store the $n \times n$ distance matrix.

Shortest Path Algorithm Comparison

Feature	Dijkstra	Bellman-Ford	Floyd-Warshall
Type	Single-Source (SSSP)	Single-Source (SSSP)	All-Pairs (APSP)
Strategy	Greedy	Dynamic Programming	Dynamic Programming
Complexity	$O(E \log V)$	$O(VE)$	$\Theta(V^3)$
Negative Edges	■ Not Allowed	■ Allowed	■ Allowed
Negative Cycles	■ Fails silently	■ Detects them	■ Fails
Space	$O(V)$	$O(V)$	$\Theta(V^2)$

■ EXAM QUESTIONS

1. Explain the Relaxation step. Write the pseudocode for $RELAX(u, v, w)$.
2. Why is Dijkstra's algorithm bounded by $O(E \log V)$ when implemented with a Binary Min-Heap? Derive the complexity.
3. State the recurrence formula used in Floyd-Warshall. What does $d(k)[i][j]$ represent?

4. Why does Bellman-Ford perform $|V|-1$ iterations? Why not $|V|$?

5. Compare Dijkstra, Bellman-Ford, and Floyd-Warshall across: type, strategy, complexity, and support for negative weights.

8. Minimum Spanning Tree (MST)

A **Minimum Spanning Tree** is a subset of edges of a connected, edge-weighted, undirected graph that **connects all vertices** together, **without any cycles**, with the **minimum possible total edge weight**.

Defining Qualities of an MST

- **Connectivity:** Must span ALL vertices in the graph.
- **Acyclic:** Must contain NO cycles — it is a tree.
- **Edge Count:** A spanning tree for $|V|$ vertices always has exactly $|V| - 1$ edges.
- **Optimality:** Minimum possible total weight among all spanning trees.
- **Adding any edge:** Introduces a unique cycle (cycle property).
- **Removing any edge:** Splits the tree into exactly two separate components (cut property).
- **Note:** An MST may not be unique if multiple edges have the same weight.

A. Kruskal's Algorithm

Grows a **forest of trees**, gradually merging them into a single spanning tree. Follows a **Greedy** strategy.

1. Sort all edges in non-decreasing order of weight.
 2. Initialize: Each vertex is its own separate component (using Disjoint-Set / Union-Find).
 3. For each edge (u, v) in sorted order:
 - If $\text{FIND}(u) \neq \text{FIND}(v)$ (they are in different components): Add edge to MST, $\text{UNION}(u, v)$.
 - Otherwise: Skip the edge (would create a cycle).
 4. Repeat until MST has $|V| - 1$ edges.
- **Data Structure:** Disjoint-Set (Union-Find) to track connected components.
 - **Time Complexity:** $O(E \log V)$ — dominated by sorting edges.

B. Prim's Algorithm

Builds the MST starting from a single **root vertex**, growing it one edge at a time. At each step it finds the '**safe edge**' — the minimum weight edge crossing the cut (connecting current tree to an outside vertex).

1. Initialize: $\text{key}[s] = 0$ (source), $\text{key}[v] = \infty$ for all others, $\pi[v] = \text{NIL}$.
 2. Place all vertices into Min-Priority Queue Q keyed by $\text{key}[v]$.
 3. While Q is not empty: Extract $u = \text{EXTRACT-MIN}(Q)$ (minimum key vertex).
 4. For each neighbor v of u : If $v \in Q$ and $w(u, v) < \text{key}[v] \rightarrow \text{update } \text{key}[v] = w(u, v), \pi[v] = u$.
 5. Repeat until Q is empty. $\text{MST} = \{(v, \pi[v]) \text{ for all } v \neq \text{source}\}$.
- **Data Structure:** Min-Priority Queue (Binary Min-Heap).
 - **Time Complexity:** $O(E \log V)$ using binary min-heap.

Feature	Kruskal's	Prim's
Strategy	Greedy (edge-based)	Greedy (vertex-based)
Approach	Grow a forest, merge components	Grow one tree from a root
Data Structure	Disjoint Set (Union-Find)	Min-Priority Queue
Complexity	$O(E \log V)$	$O(E \log V)$ with Binary Heap
Better for	Sparse graphs	Dense graphs

■ EXAM QUESTIONS

1. State all 4 defining qualities that a subgraph must satisfy to be considered a Minimum Spanning Tree.

2. Explain Kruskal's algorithm step by step. What data structure does it use and why?
3. Trace Prim's algorithm from a given source vertex on a weighted graph.
4. What is the time complexity of Prim's algorithm with a binary min-heap? Derive it.

9. Knapsack Problem

A thief must choose items with specific weights and values to **maximize profit** while staying within a total weight capacity **W**. Two variants exist:

A. Fractional Knapsack (Greedy)

Items **can be divided** — take any fraction of an item.

1. Calculate value-per-unit-weight (v_i/w_i) for every item.
2. Sort items in descending order of value-per-unit-weight.
3. Take as much as possible of the highest-ratio item.
4. Continue until weight limit W is reached (take fraction of last item if needed).

- **Optimality:** Greedy sorting by value/weight is guaranteed optimal.
- **Time Complexity:** $O(n \log n)$ — dominated by sorting.

B. 0/1 Knapsack (Dynamic Programming)

Items are **indivisible** — either take the entire item (1) or leave it (0). Greedy fails here because taking the highest value/weight item may leave unusable space.

Build a DP table where $dp[k][w]$ = maximum benefit using items $\{1, \dots, k\}$ with capacity w :

$$dp[k][w] = dp[k-1][w] \text{ if } w_k > w \text{ (cannot include item } k) = \max(dp[k-1][w], \text{ if } w_k \leq w \text{ } dp[k-1][w - w_k] + v_k)$$

1. Create $(n+1) \times (W+1)$ table, initialized to 0.
 2. For each item k (1 to n): For each capacity w (0 to W):
 - If $w_k > w$: $dp[k][w] = dp[k-1][w]$ (skip item k).
 - Else: $dp[k][w] = \max(dp[k-1][w], dp[k-1][w - w_k] + v_k)$.
 3. Optimal value = $dp[n][W]$. Backtrack through the table to find selected items.
- **Time Complexity:** $O(nW)$ — n items $\times$ W capacity.
 - **Space Complexity:** $O(nW)$ — DP table size.

Feature	Fractional Knapsack	0/1 Knapsack
Divisibility	Items can be cut/fractioned	Items are all-or-nothing
Approach	Greedy	Dynamic Programming
Optimality via Sorting	■ Always optimal	■ Sorting is NOT optimal
Time Complexity	$O(n \log n)$	$O(nW)$
Space Complexity	$O(1)$ extra	$O(nW)$

■ EXAM QUESTIONS

1. Explain why a Greedy strategy fails for the 0/1 Knapsack problem. Give a counterexample.
2. Write the DP recurrence for the 0/1 Knapsack problem. What does $dp[k][w]$ represent?

3. What is the time complexity of the 0/1 Knapsack DP solution?

10. Matrix Chain Multiplication

The goal is NOT to perform the matrix multiplication, but to determine the most efficient **parenthesization order** to minimize the total number of **scalar multiplications**.

Core Concepts

- **Associativity:** $(A \times B) \times C = A \times (B \times C)$ — result is the same, but the cost can differ drastically.
- **Cost of Multiplication:** Multiplying a $[p \times q]$ matrix by a $[q \times r]$ matrix requires $p \times q \times r$ scalar multiplications, yielding a $[p \times r]$ result.
- **Optimal Substructure:** The optimal parenthesization of $A_1 \dots A_j$ splits at some k , optimally parenthesizing both sub-chains independently.

Recursive Formula (DP Recurrence)

$$m[i][j] = 0 \text{ if } i == j = \min \text{ over } i \leq k < j \text{ of: } \{ m[i][k] + m[k+1][j] + p(i-1) \times p(k) \times p(j) \}$$

Where $p_{i-1} \times p_i$ is the dimension of matrix A_i . The $s[i][j]$ table records the optimal split point k for reconstruction.

Algorithm Steps

1. Let $p[]$ store dimensions: matrix A_i has dimensions $p[i-1] \times p[i]$.
 2. Initialize: $m[i][i] = 0$ for all i (single matrix, no multiplication needed).
 3. Fill table m by increasing chain length l (from 2 to n).
 4. For each chain $A_i \dots A_j$ of length l : Try all split points k from i to $j-1$.
Compute cost = $m[i][k] + m[k+1][j] + p[i-1] \times p[k] \times p[j]$. Update $m[i][j]$ if smaller.
 5. $m[1][n]$ = minimum cost. Backtrack using s table to find optimal parenthesization.
- **Time Complexity:** $O(n^3)$ — Three nested loops (chain length, start index, split point).
 - **Space Complexity:** $\Theta(n^2)$ — $m[][]$ and $s[][]$ tables.

■ EXAM QUESTIONS

1. What is the objective of Matrix Chain Multiplication? Why don't we simply multiply left-to-right?
2. Write the DP recurrence for $m[i][j]$. What do each of the terms represent?
3. What is the time and space complexity of the Matrix Chain Multiplication DP solution?

11. Traveling Salesperson Problem (TSP)

TSP seeks the **shortest possible route** that visits every city **exactly once** and returns to the starting city — a **Hamiltonian Cycle** of minimum total weight.

A. Dynamic Programming Approach

Build optimal paths over subsets of cities using previously computed sub-solutions.

State Definition: $g(i, S)$ = length of the shortest path starting at city i , visiting all cities in set S , ending at city 1.

$$g(i, S) = \min \text{ over } j \text{ in } S \text{ of: } \{ c(i, j) + g(j, S - \{j\}) \} \text{ Base case: } g(i, \{1\}) = c(i, 1) \text{ (direct return to start)}$$

- c_{ij} = distance/cost from city i to city j .
- **Time Complexity:** $O(2^n \times n^2)$ — exponential, but better than $n!$ brute force.

B. Branch and Bound (B&B;) Approach

An optimization over exhaustive search that uses **pruning** to eliminate paths that cannot lead to a better solution than the current best known.

- **Reduced Cost Matrix:** Reduce every row and column so each has at least one zero. The sum of values subtracted = Lower Bound for that node.
- **Generate Children:** For each city that can be visited next, create a child node.
- **Least Cost First:** Always expand the node with the smallest lower bound (priority queue).
- **Pruning:** If a node's lower bound exceeds the current best solution, discard it — it cannot improve the answer.
- **Backtracking:** Continue level-order until a leaf node (complete tour) is found with minimum cost.

Method	Approach Type	Use Case
Backtracking	Exhaustive Search	Finding all permutations — not optimal for large n .
Dynamic Programming	Subproblem Reuse	Guarantees optimal; effective for smaller city sets.
Branch and Bound	Level-Order Expansion	Efficiently finds optimal by pruning non-promising branches.

■ EXAM QUESTIONS

1. Define the TSP problem. What makes it NP-Hard?
2. Write the DP recurrence for TSP. What does $g(i, S)$ represent?
3. Explain the Branch and Bound strategy for TSP. What is the role of the Reduced Cost Matrix?

UNIT IV — STRING MATCHING, BACKTRACKING & NP PROBLEMS

12. String Matching — Boyer-Moore Algorithm

The general string matching problem: find all occurrences of pattern **P** (length m) in text **T** (length n). Boyer-Moore is a highly efficient algorithm that shifts the pattern window **left-to-right** but compares characters **right-to-left**.

Two Shift Rules

At each mismatch, Boyer-Moore calculates two possible shifts and takes the **maximum**:

- **Bad Character Rule** — based on the mismatched character in the text.
- **Good Suffix Rule** — based on the matched suffix that preceded the mismatch.

Bad Character Rule — 4 Cases

When a mismatch occurs at text character **X** aligned with pattern position j :

Case	Condition	Action / Shift
Case 1	Character X does NOT appear anywhere in pattern P	Shift = m (skip entire pattern window)
Case 2	X appears in P, but NOT as the last character. Align with rightmost occurrence of X.	Shift = $m - 1 - j$ (where j is rightmost index of X in P)
Case 3	X is the LAST character of P but NOT in first $m-1$ chars	Shift = m
Case 4	X is the LAST character AND appears in first $m-1$ chars	Align with rightmost occurrence in the prefix of P

Bad Character Shift (Case 2): Shift = $m - 1 - j$ (where j = index of rightmost occurrence of X in pattern P)

Time Complexity

- **Best Case:** $O(n/m)$ — sub-linear when pattern doesn't appear frequently.
- **Worst Case (pattern not found):** $O(n + m)$.
- **Worst Case (pattern occurs many times):** $O(nm)$.

■ EXAM QUESTIONS

1. In Boyer-Moore, what are the 4 cases of the Bad Character rule? Give the shift formula.
2. Why does Boyer-Moore compare characters right-to-left even though it scans left-to-right?
3. When should you use Good Suffix vs Bad Character rule in Boyer-Moore?

13. String Matching — Knuth-Morris-Pratt (KMP) Algorithm

KMP avoids redundant comparisons by **preprocessing the pattern P** to build a **Failure Function $f(j)$** . When a mismatch occurs, instead of restarting from scratch, it uses the failure function to skip already-matched characters.

The Failure Function $f(j)$

Definition: $f(j)$ = the length of the longest proper prefix of $P[1..j]$ that is also a suffix of $P[1..j]$.

If $P[j] == P[i]$: $f(i) = j + 1$ (extend the matching prefix) If mismatch: $j = f(j - 1)$ (jump back using failure function) (i does NOT move backward – key efficiency gain)

Why KMP is $O(m + n)$ — Linear Time Analysis

- **Preprocessing Phase:** Build failure function in $O(m)$ — scans pattern once.
- **Search Phase:** The text pointer i NEVER moves backward → scans text T in $O(n)$.
- **Pattern pointer j :** Can only decrease when using $f(j-1)$, but total increases $\leq n$ → amortized $O(n)$.
- **Total Time Complexity:** $O(m + n)$ — strictly linear.

Example: Computing Failure Function

For pattern $P = A B A B C$:

Index j	1	2	3	4	5
Pattern $P[j]$	A	B	A	B	C
$f(j)$	0	0	1	2	0

■ EXAM QUESTIONS

1. Define the failure function $f(j)$ in KMP. Why is it called the 'failure' function?
2. How does KMP achieve linear $O(m+n)$ time? Why doesn't the text pointer ever move backward?
3. Compute the failure function for pattern $P = 'AABAAC'$.

14. Backtracking & Recursive Backtracking

Backtracking is a form of **recursion** that maps to a **Depth-First Search** of an abstract state-space tree. It systematically explores all possible solutions by trying a path, recognizing dead ends early, and undoing choices to try alternatives.

Core Concepts

- **State-Space Tree:** An abstract tree where each node represents a partial decision state.
- **Choose → Explore → Unchoose:** Make a choice, recurse deeper, then revoke the choice and try the next option.
- **Promising Node:** A node that can potentially lead to a valid complete solution — worth exploring.
- **Non-Promising Node:** A node that cannot possibly lead to a solution — dead end, should be pruned.
- **Pruning:** The act of checking a node, recognizing it as non-promising, and immediately backtracking to the parent — avoiding wasteful exploration of its entire subtree.
- **Bad Leaf:** A leaf node that represents a dead end (no valid solution).

Generic Backtracking Pseudocode

```
checknode(v): if promising(v): if is_solution(v): write solution else: for each
child u of v: checknode(u)
```

Application: N-Queens Problem

Objective: Place N queens on an $N \times N$ chessboard such that no two queens attack each other (no shared row, column, or diagonal).

- Place queens row by row (one per row).

- At each row, try placing in each column (1 to N).
- promising(v): Check if current queen placement conflicts with any previously placed queen.
- If promising → recurse to next row. If not → backtrack (try next column).
- If all N queens placed successfully → record solution.

■ EXAM QUESTIONS

1. Define 'Promising node' and 'Pruning' in the context of Backtracking.
2. Write the pseudocode for the generic recursive backtracking (checknode) function.
3. Outline the N-Queens solution using backtracking. What does the promising() function check?

15. Complexity Measures & NP Problems

Complexity theory classifies problems by how the time or space required grows with input size, determining whether problems are **tractable** (solvable efficiently) or **intractable** (computationally infeasible).

Polynomial vs Non-Polynomial Time

- **Polynomial Time:** $O(n^2)$, $O(n^3)$, $O(n^k)$ — tractable, feasible for large inputs.
- **Non-Polynomial (Exponential):** $O(2^n)$, $O(n!)$ — intractable, infeasible for large n .
- **Rule of thumb:** Polynomial $\equiv$ 'efficiently solvable'.

Complexity Classes — Formal Definitions

Class P	Decision problems solvable by a deterministic algorithm in polynomial time $O(n^k)$. Example: Sorting, Shortest Path, MST.
Class NP (Nondeterministic Polynomial)	Decision problems verifiable in polynomial time, given a 'certificate' (guessed solution). Uses a 2-stage model: (1) Non-deterministic guessing stage — generate a certificate. (2) Deterministic verification stage — check if certificate is valid in poly-time. Example: Boolean SAT, Hamiltonian Cycle.
NP-Hard	Problems at LEAST as hard as the hardest NP problems. Every NP problem can be reduced to an NP-Hard problem in polynomial time. NP-Hard problems may NOT be in NP (may not even be decision problems). Examples: TSP (optimization), Knapsack, Graph Coloring, Halting Problem.
NP-Complete	The HARDEST problems strictly INSIDE NP class. A problem X is NP-Complete if: (1) $X \in \text{NP}$ (can be verified in poly-time), AND (2) X is NP-Hard (every NP problem reduces to X in poly-time). Examples: SAT, 3-SAT, Hamiltonian Cycle, Vertex Cover, Clique.

Approximation Algorithms

For computationally intractable (NP-Hard) problems, approximation algorithms provide solutions **close to optimal** in polynomial time. Quality is measured by the **approximation ratio** $p(n)$: the ratio of approximate solution cost to optimal cost.

Boolean Satisfiability Problem (SAT)

Problem: Given a Boolean formula in **Conjunctive Normal Form (CNF)** — a conjunction (AND) of clauses where each clause is a disjunction (OR) of literals — determine if there exists an assignment of TRUE/FALSE values to variables that makes the entire formula TRUE.

Example CNF formula:

Formula: (X1 OR X3) AND (NOT X2 OR NOT X3) Try: X1 = TRUE, X2 = FALSE, X3 = TRUE
Clause 1: (TRUE OR TRUE) = TRUE Clause 2: (TRUE OR FALSE) = TRUE Formula = TRUE ← SATISFIABLE!

- **SAT is NP-Complete** (Cook's Theorem, 1971 — the first problem proven NP-Complete).
- **3-SAT**: SAT restricted to exactly 3 literals per clause — also NP-Complete.

NP Classes — Summary

Class	Definition	Verifiable?	Solvable in Poly-time?	Examples
P	Solvable in poly-time	Yes (trivially)	Yes	Sorting, SSSP, MST
NP	Verifiable in poly-time	Yes	Unknown	SAT, 3-SAT, HC
NP-Hard	At least as hard as NP	Not necessarily	No (likely)	TSP opt, Halting
NP-Complete	In NP AND NP-Hard	Yes	No (likely)	SAT, 3-SAT, VC

■ EXAM QUESTIONS

1. Differentiate between Polynomial and Non-polynomial time complexity with examples.
2. Formally define NP-Hard and NP-Complete. How do they relate to each other? Draw the Venn diagram.
3. What is the Boolean Satisfiability (SAT) problem? To which complexity class does it belong and why?
4. What is an approximation algorithm? What is the approximation ratio?

■ QUICK REVISION MODULE

Key Definitions — Must Know

Strongly Connected Component (SCC)	A DIRECTED maximally connected subgraph where paths $u \rightarrow v$ and $v \rightarrow u$ exist for every pair.
Articulation Point (AP)	A node in an undirected graph that, if removed, splits the graph into multiple disconnected components.
Topological Sort	A linear ordering of vertices in a DAG such that u appears before v for every directed edge (u,v) .
Relaxation	Updating $v.d$ if the new path via u is shorter: if $v.d > u.d + w(u,v)$ then $v.d = u.d + w(u,v)$.
Backtracking	A recursive DFS methodology — choose, explore, unchoose — pruning non-promising paths to search state-space trees efficiently.
Class P	Decision problems solvable by a deterministic algorithm in polynomial time $O(n^k)$.
Class NP	Decision problems verifiable in polynomial time given a guessed certificate (2-stage: guess + verify).
NP-Complete	A problem that is BOTH in NP AND is NP-Hard — the hardest problems inside the NP class.
Safe Edge (MST)	An edge that can be added to a partial MST without violating MST properties — always a minimum weight crossing edge.
Promising Node	A backtracking node that can potentially lead to a valid solution — worth exploring further.

Crucial Formulas — Must Memorize

Adjacency Matrix Entry	$a(i,j) = 1$ if $(i,j) \in E$, else $a(i,j) = 0$
Articulation Point Low Value	$Low(w) = \min \{ d[w], \min\{Low[child[w]]\}, \min\{d[back-edge\ endpoints]\} \}$
Relaxation Logic	$if (v.d > u.d + w(u,v)) \rightarrow v.d = u.d + w(u,v); v.\pi = u$
Floyd-Warshall DP Recurrence	$d(k)[i][j] = \min(d(k-1)[i][j], d(k-1)[i][k] + d(k-1)[k][j])$
Bellman-Ford Neg Cycle Check	After $ V -1$ passes, relax once more. If any $v.d$ decreases $\rightarrow$ NEGATIVE CYCLE exists
KMP Failure Function Assignment	If $P[j] == P[i]: f(i) = j+1$ On mismatch: $j = f(j-1)$

Boyer-Moore Bad Char Shift	$\text{Shift} = m - 1 - j$ (j = rightmost index of mismatched char in P)
0/1 Knapsack DP Recurrence	$\text{dp}[k][w] = \max(\text{dp}[k-1][w], \text{dp}[k-1][w-w_k] + v_k) \text{ if } w_k \leq w$
Matrix Chain Mult Recurrence	$m[i][j] = \min \text{ over } k: \{ m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j] \}$
TSP DP Recurrence	$g(i, S) = \min \text{ over } j \in S: \{ c(i, j) + g(j, S - \{j\}) \}$
BFS/DFS Complexity	$O(V + E)$
Dijkstra (Binary Heap)	$O(E \log V) = V \times O(\log V) \text{ [extract-min]} + E \times O(\log V) \text{ [decrease-key]}$

■ Key Time Complexities — Master Table

Algorithm / Operation	Time Complexity	Space	Notes
BFS	$O(V + E)$	$O(V)$	FIFO Queue
DFS / Topological Sort	$O(V + E) [\Theta(V+E)]$	$O(V)$	Recursive / Stack
SCC (Kosaraju's)	$O(V + E)$	$O(V)$	Two DFS passes
Dijkstra (Array)	$O(V^2)$	$O(V)$	Dense graphs
Dijkstra (Binary Heap)	$O(E \log V)$	$O(V)$	Sparse graphs
Dijkstra (Fibonacci Heap)	$O(E + V \log V)$	$O(V)$	Theoretical best
Bellman-Ford	$O(VE)$	$O(V)$	Handles negative weights
Floyd-Warshall	$\Theta(V^3)$	$\Theta(V^2)$	All-pairs
Kruskal's MST	$O(E \log V)$	$O(V)$	Union-Find
Prim's MST (Binary Heap)	$O(E \log V)$	$O(V)$	Priority queue
Fractional Knapsack	$O(n \log n)$	$O(1)$	Greedy + sort
0/1 Knapsack (DP)	$O(nW)$	$O(nW)$	n items, W capacity
Matrix Chain Mult	$O(n^3)$	$\Theta(n^2)$	DP table
TSP (DP)	$O(2^n \times n^2)$	$O(2^n \times n)$	Exponential
Boyer-Moore (worst)	$O(nm)$	$O(m + \Sigma)$	Pattern occurs often
Boyer-Moore (no match)	$O(n + m)$	$O(m + \Sigma)$	Best case $O(n/m)$
KMP	$O(n + m)$	$O(m)$	Linear time

■ Algorithm Strategy Summary

Algorithm	Problem Type	Strategy	Key Data Structure
Dijkstra	SSSP	Greedy	Min-Priority Queue

Bellman-Ford	SSSP + Neg Weights	Dynamic Programming	Edge List
Floyd-Warshall	All-Pairs SP	Dynamic Programming	2D Matrix
Kruskal's	MST	Greedy	Disjoint Set (Union-Find)
Prim's	MST	Greedy	Min-Priority Queue
Fractional Knapsack	Optimization	Greedy	Sorted Array
0/1 Knapsack	Optimization	Dynamic Programming	2D DP Table
Matrix Chain Mult	Optimization	Dynamic Programming	2D DP Table
TSP (DP)	Optimization	Dynamic Programming	Subset DP Table
TSP (B&B)	Optimization	Branch and Bound	Priority Queue
Boyer-Moore	String Matching	Right-to-Left Scan	Bad Char Table
KMP	String Matching	Prefix Matching	Failure Function Array
Backtracking	Combinatorial Search	DFS + Pruning	Recursion Stack
SCC	Graph Property	Two-pass DFS	Stack / Transpose Graph
Topological Sort	DAG Ordering	DFS Finishing Time	Linked List

■ **Good luck with your exam and viva! Remember: Understand the intuition behind each algorithm, not just the steps. Know when to apply each algorithm based on graph type and constraints. Practice tracing through examples manually — examiners love step-by-step traces.**